
Transaction Attributes

Transaction attributes define **how a transaction behaves** during execution. In Spring, they are mainly configured using the `@Transactional` annotation.

1 Propagation

Definition:

Defines **how a transaction behaves when one transaction calls another**.

Common Types

Propagation	Meaning	Real-Time Example
REQUIRED (default)	Join existing or create new	Order + Payment
REQUIRES_NEW	Always create new	Logging
SUPPORTS	Use if exists	Read-only method
MANDATORY	Must have existing	Security check
NEVER	Must not have	Non-transactional audit
NOT_SUPPORTED	Suspend existing	Reporting
NESTED	Savepoint	Partial rollback

Example:

`@Transactional(propagation = Propagation.REQUIRED)`

1.Propagation (How transactions interact)

REQUIRED (Default)

Real Time:

Order Service → Payment Service

- If order already started a transaction → payment joins it
- If payment fails → order is rolled back

Most commonly used

REQUIRES_NEW

Real Time:

Order processing → Audit logging

- Order fails
- Audit log must still be saved

Independent transaction

Order rollback does NOT affect logs

SUPPORTS

Real Time:

View product details

- Works with or without a transaction
 - Read-only operations
-

MANDATORY

Real Time:

Security check before fund transfer

- Must run inside an existing transaction
 - If no transaction → error
-

NEVER

Real Time:

System health check

- Must not run inside a transaction
- Throws exception if transaction exists

NOT_SUPPORTED

Real Time:

Monthly report generation

- Suspends current transaction
- Avoids locking large tables

NESTED

Real Time:

Bulk order processing

- Order #3 fails
- Orders #1 and #2 succeed

Uses savepoints

Partial rollback

2.Isolation

Definition:

Controls **data visibility between concurrent transactions.**

Isolation Levels

Level	Prevents
READ_UNCOMMITTED	None
READ_COMMITTED	Dirty reads
REPEATABLE_READ	Dirty + Non-repeatable
SERIALIZABLE	All issues

Example:

@Transactional(isolation = Isolation.READ_COMMITTED)

Isolation (Concurrent user behavior)

READ_UNCOMMITTED

Real Time:

- User sees uncommitted balance (wrong data)
Rarely used
-

READ_COMMITTED

Real Time:

ATM withdrawal

- Balance is read only after commit
Prevents dirty reads
-

REPEATABLE_READ

Real Time:

E-commerce cart

- Price remains same during checkout
Prevents price change during transaction
-

SERIALIZABLE

Real Time:

Seat reservation system

- One user books at a time
Highest consistency
Slow performance

3.Read-Only

Real Time:

Search products / View history

- No insert/update allowed
- Faster performance

@Transactional(readOnly = true)

3 Read-Only

Definition:

Optimizes performance for **read-only operations**.

Example:

@Transactional(readOnly = true)

Improves performance
Prevents accidental updates

4 Timeout

Definition:

Maximum time (in seconds) a transaction can run before rollback.

Example:

@Transactional(timeout = 10)

Real Time:

Online payment gateway

- If payment takes > 10 seconds
- Transaction is cancelled automatically

Prevents deadlocks

Improves system stability

5.Rollback Rules

Definition:

Controls **when a transaction should rollback**.

Default Behavior

- Rolls back on RuntimeException
- Does NOT rollback on checked exceptions

Real Time:

- RuntimeException → rollback
- Checked Exception → commit

Custom Rollback

@Transactional(rollbackFor = Exception.class)

@Transactional(noRollbackFor = ArithmeticException.class)

Full Example

```
@Transactional(  
    propagation = Propagation.REQUIRED,  
    isolation = Isolation.READ_COMMITTED,  
    timeout = 10,  
    readOnly = false,  
    rollbackFor = Exception.class  
)  
public void transferMoney() {  
    debit();  
}
```

```
    credit();  
}
```

Summary Table

Attribute	Purpose
Propagation	Transaction relationship
Isolation	Concurrency control
Read-Only	Performance
Timeout	Execution limit
Rollback	Failure handling

Transaction attributes define how, when, and under what conditions a transaction executes and rolls back.
